# Real-Time Intrusion Detection for IoT Networks: Evaluating ML/DL Models and Feasibility of Edge Deployment on the Kria KV260

Submitted by

Willow Connelly

Computer Science

To

The Donna and Walt Young Honors College

Oakland University

In partial fulfillment of the

requirement to graduate from

The Donna and Walt Young Honors College

Mentor: Katherine Bowers, Ph.D.

Department of Electrical and Computer Engineering

Oakland University

December 05, 2025

**Abstract**

The Internet of Things (IoT) is rapidly revolutionizing human life across numerous sectors, such as pacemakers in healthcare, soil nutrient sensors in agriculture, and smart cameras in consumer applications. Along with this versatility comes the difficulty of defining a singular security solution, as each device has its own protection measures or lack thereof. Attacks on the system can corrupt device data, disable or modify the application, or even give unauthorized access to the network's servers. To address these challenges, this project investigates the design of a real-time intrusion detection system (IDS) that alerts the server toward a potential attack. This IDS was then optimized for deployment at the network edge to act as a guardian of the IoT system. Both machine learning and deep learning methods were assessed, namely Random Forest (RF), Extreme Gradient Boosting (XGBoost), and a Convolutional Neural Network (CNN) with and without self-attention layers. Benchmarking was done using accuracy, F1 score, and other key metrics to provide a quantitative evaluation. By balancing efficiency and complexity, this study aims to advance IDS technology for practical, IoT environments, improving their resilience against emerging threats.

# 1 Introduction

The Internet of Things (IoT) refers to a network of interconnected, task-specific physical devices, or "things", such as sensors and embedded systems that communicate online [1]. These devices operate across numerous domains, including smart homes, industrial automation, and connected vehicles equipped with sensors and on-board computers [2] where they gather, transmit, and respond to local environmental data. Over the years, the prevalence of IoT has exploded globally; total IoT connections are forecast to more than double from 19.8 billion in 2025 to over 40 billion by 2034 [3]. As much benefit as IoT brings to peoples' daily lives, its ubiquity simultaneously introduces both old and emerging security threats.

Because IoT spans a wide range of applications, the diversity of device types makes it difficult to define a singular, uniform security solution. Moreover, these heterogeneous devices often have strict resource and energy constraints, exacerbating the difficulty of security design. As a result, this makes them particularly susceptible to cyber-attacks. Since these networks frequently handle sensitive and personal information, compromises can yield severe consequences [4]. For instance, in the healthcare sector, attackers could exploit weak authentication protocols in sensor devices, such as blood pressure or heart-rate devices, which could grant them further access within hospital networks. Such access could enable adversaries to alter hospital systems or access patient information governed by the Health Insurance Portability and Accountability Act (HIPAA). Beyond privacy violations, unauthorized modification and manipulation of medical IoT devices could even endanger patient life and safety [5].

Attack surfaces in IoT systems span multiple levels, including but not limited to the physical, sensing layer (e.g., Booting or sleep deprivation attacks), the network layer (e.g., Dos/DDos or sniffing), and the gateway layer (e.g., man-in-the-middle attacks or compromised end-to-end encryption) [6]. This work focuses specifically on attacks on the network-layer.

Intrusion detection systems (IDS) have become a widely researched defensive solution

for detecting malicious activity [7] in an IoT landscape. However, IDS design is non-trivial, requiring many facets to be considered, such as resource consumption and detection delay, before applying the most effective solution. Existing IDS research commonly employs one of two architectures: machine learning (ML) models or deep learning (DL) models. Each has benefits and limitations. ML architectures offer lightweight and often faster training and inference but may lack the complexity necessary for detecting existing or novel anomalies. In contrast, DL models can learn hidden patterns and analyze high-dimensional data but are frequently too computationally heavy for IoT hardware. Compounding this challenge, much of the literature evaluates IDS models solely in offline or high-resource environments, leaving testing in real-world deployment scenarios underexplored [8, 9].

Additionally, many IDS implementations perform only binary classification, labeling traffic as malicious or benign. Though useful, this limits the quality of information produced and restricts the granularity of potential response actions. By incorporating multi-class classification, an IDS can determine the specific attack type, enabling more accurate and rapid mitigation strategies that reduce damage and improve overall system security.

To address these challenges, this paper investigates the feasibility of designing an AI-based intrusion detection model suitable for resource-limited edge devices in an IoT gateway. Both ML and DL models are analyzed quantitatively to compare their respective trade-offs and to help inform future IDS design for IoT environments. Furthermore, to provide more actionable insights, the proposed IDS classifies network traffic flows as benign or malicious and further distinguishes them into specific attack or benign traffic classes. The RT-IoT2022 dataset [10] is used to evaluate four model architectures: two ML models, Random Forest (RF) and XGBoost, and two DL convolutional neural network (CNN) models, one incorporating self-attention and one without. The aim of this study is to identify a model that achieves high detection accuracy and recall while maintaining low computational overhead. The most suitable models are subsequently optimized and, if applicable, quantized to prepare for use on an edge device acting as a network gateway. This deployment enables real-time

inference with minimal latency times. The overarching objective is to advance IDS design for deployment at the edge to strengthen IoT security through more efficient detection.

The main contributions of this work are outlined as follows:

- Developing and training several ML and DL models on a recent IoT intrusion detection dataset to establish a robust foundation for assessing how different architectures perform on classifying network traffic.
- Evaluating and comparing the performance of Random Forest, XGBoost, and CNNs (with and without attention) using key performance metrics to reveal how different AI mechanisms influence network flow classification and identify which approach most effectively balances computational cost and performance for real-world IDS applications, a critical consideration in resource-constrained IoT environments.
- Determining the feasibility of deploying the optimized model on an edge device to demonstrate its capability for real-time inference, thereby illustrating how running an IDS closer to the network can reduce latency and enhance overall security.

The remainder of this paper is structured accordingly. The state of related literature is reviewed in Section 2. Section 3 describes the background of the methods described in Section 4 performed in this research. Sections 5 and 6 present the execution of the experiments and corresponding results, respectively. Finally, Section 7 discusses the limitations and future directions for this work, and Section 8 summarizes the key takeaways.

## 2 Related Works

In IDSs, an increasingly popular approach is to employ ML and DL algorithms to classify network traffic. Ahmed et al. [11] investigated the efficacy of ML ensemble models for IDSs, comparing algorithms, such as RF and XGBoost trained on the RT-IoT2022 dataset [10]. Although each classifier performed well, XGBoost and LightGBM achieved the best accuracy,

up to 4% higher than the other models tested. However, these results were limited to a single dataset and require additional validation. The studies in [12, 13] mitigate this issue by training ensemble ML models on multiple datasets. Specifically, [13] focused on detecting threats in cyber-physical systems (CPS) and designed a custom ML ensemble model composed of five base learners. Four ensemble strategies were evaluated, with boosting yielding the most significant improvement, confirming the work in the previous study. Accuracy improved by 8–10% compared to the individual base models. Both this and the framework developed in [14] addressed the need for updatable models that evolve alongside the attack space. The proposed framework in [14] begins with generating the dataset and ends with outputting a trained and evaluated binary classifier to serve as an IDS. After testing five ML models, including Decision Trees (DT) and RF, in two separate environments, DT achieved the best overall performance with an F1-Score of 96%. This study considered not only accuracy metrics, but also resource usage, of which the DT scored well in as well.

Ensemble methods have not only been successful in ML contexts, but also in federated learning (FL) and DL domains. One such study employed ensemble multi-view FL for intrusion detection [15]. Three DL models, each trained on a separate view of the data, were distributed to IoT devices, where they were updated locally and then fused together using the voting ensemble technique. Compared to non-FL methods, this design improved both accuracy and long-term adaptability while also introducing an additional layer of security inherent to FL. Another DL-based approach applied ensemble learning to multi-label classification (MLC) [16]. The authors designed an ensemble of neural networks that sequentially learned the distribution of a single set of labels, with each network ingesting the parameters from the preceding model. This design achieved a higher performance than other MLC algorithms without increasing computational complexity. Similarly, Fares et al. [17] explored methods to reduce the consumption of computational resources by designing a Transformer architecture that incorporated Kolmogorov-Arnold Networks. These networks utilize learnable univariate function components instead of traditional multilayer perceptron layers that

have fixed weights. This significantly reduced the parameter count while maintaining high accuracy, achieving approximately 99% on IoT datasets. A different strategy for reducing parameters was proposed in [18] which focused on optimizing the feature set rather than the model architecture. The authors introduced a feature extraction method that combined principal component analysis, Hotelling's $T^2$, and a convolutional Bi-directional Long-Short Term Memory (BiLSTM) classifier. This approach effectively reduced redundancy and feature parameters, further enhancing detection efficiency.

Other non-ensemble approaches have also demonstrated strong performance in IoT-based IDSs. Several studies examined and compared the effectiveness of various DL algorithms [19–21]. Given that the data contained temporal information, these works found Bi-LSTMS particularly effective for intrusion detection. CNNs also showed promise in [21], where the authors suggested that combining a CNN's ability to identify spatial patterns with an LSTM's temporal analysis capabilities could further enhance detection performance. This idea was explored in [22–24], all of which designed a hybrid CNN-LSTM IDS that stacked a CNN onto a LSTM. In these designs, the CNN first extracted spatial feature representations, which were then processed by the LSTM to identify temporal dependencies. Both studies reported high rates of accuracy, averaging above 93% and 97%, respectively, across datasets. Bassam et al. [25] extended this concept by developing a more complex hybrid IDS, integrating four different neural network algorithms with dense layers added. Each model was trained independently with their output feature maps concatenated and then condensed via another dense layer before final classification. This architecture leveraged the unique strengths of each model, such as spatial pattern extraction, temporal dependency modeling, long-term, efficient data management, and critical feature prioritization. This model outperformed a similar ML-based hybrid model, along with other existing DL structures found in the current literature. This showcases hybrid DL model's effectiveness in IoT environments.

While many IDS studies focus on improving model accuracy and efficiency, the question of real-world deployment remains insufficiently explored [8, 9]. In practical IoT environ-

ments, these designs must operate in real-time and remain lightweight enough for resource-constrained edge devices. By performing most computations locally, edge-based IDSs can reduce bandwidth overhead and latency. However, only a limited number of studies have directly addressed this challenge. For instance, Singh [26] generated a custom dataset containing benign and malicious traffic to train ML models for edge-based IDSs. Among the various ML models tested, the DT model had the best performance. However, the study did not cover real-world deployment or hardware-level validation. A related study in the Internet of Vehicles (IoV) domain [27] explored a more realistic deployment scenario by testing several DL architectures on a Raspberry Pi 3B device acting as a Multi-access Edge Computing server. The models were trained on a server and then run and tested at the edge. A CNN model achieved the fastest run time among the tested architectures.

Current research demonstrates the growing use of artificial intelligence techniques for intrusion detection in IoT environments [7, 9]. While some studies emphasize the use of traditional ML-based approaches [11, 12, 14] and others explore DL architectures [19–21], both have unique advantages, and this work conducts a direct comparison between the two. Although several of the studies mentioned have examined resource and time consumption [14, 16–18], this aspect remains a critical consideration for ensuring feasibility in real-world, resource-constrained IoT settings and warrants further attention. Existing work has also seldom validated IDS models through results based on resource performance or hardware-based deployment [8,9]. To address these gaps, this paper focuses on developing lightweight, real-time detection models that can function as a fast and accurate edge gateway into the IoT networks. By prioritizing both accuracy and resource efficiency, this research contributes toward advancing IDS design for practical and scalable deployment in environments that handle sensitive or industrial data.

# 3 Background

## 3.1 RT-IoT2022 Dataset

This project begins with the examination of the RT-IoT2022 dataset to identify key features relevant to network traffic classification along with determining an appropriate modeling structure. The RT-IoT2022 dataset, containing over 120,000 entries and 83 features, was selected for its diverse attack types and recency [11,25]. Furthermore, the dataset represents real-world IoT traffic data, which is required for this research to determine real-world IoT IDS feasibility. The dataset was derived from a real-time IoT environment with simulated attack scenarios [10]. Each entry represents a bidirectional network flow rather than a raw packet capture, enabling faster preprocessing through reduced data granularity. The dataset includes twelve target classes—three benign and nine attack types, such as ARP poisoning, DoS/DDoS attacks, and network mapper (NMAP) scan patterns. The data was gathered from three IoT devices, ThingSpeak-LED, Wipro-Bulb, and MQTT-Temp. Given the three-month timeline of this project, selecting a relatively clean and manageable dataset was essential.

## 3.2 ML Models

Four AI models are utilized to evaluate classification on the RT-IoT2022 dataset. Of these, two are ML ensemble models, RF and XGBoost, and two are DL CNN models, one of which incorporates attention layers.

### 3.2.1 Random Forest

The RF model consists of an ensemble of independent Decision Trees (DT) using the bagging strategy, where majority voting among the trees determines the final prediction. This method reduces overfitting by training each tree on a random subset of the data, helping them to capture diverse patterns and improve generalization [28]. Along with this, RF was chosen for

its compatibility with structured, tabular data [29], which aligns with RT-IoT2022's format.

### 3.2.2 XGBoost

XGBoost, the second ensemble ML model, also employs DTs but uses gradient boosting instead of bagging. In this approach, trees are built sequentially, with each new tree adjusting to minimize the errors of the previous tree. XGBoost is a powerful algorithm that reduces overfitting and increases accuracy through regularization techniques [30], making it a strong candidate for efficient, real-time IDS applications.

## 3.3 DL Models

### 3.3.1 CNN

The baseline CNN model is a neural network architecture that employs convolutional and pooling layers to identify hidden patterns in the data. Commonly, two-dimensional CNNs (2D-CNN) are used to process image data or other 2D inputs. In this paper however, a 2D-CNN architecture is applied to tabular network flow data. This is done for compatibility with the Kria KV260 hardware and Vitis-AI framework by reshaping the input feature vector to include a dummy spatial height dimension. Structurally, CNNs are composed of three main components: convolutional layers, pooling layers, and dense, fully-connected layers. Convolutional layers extract local structures to build complex feature maps by applying learnable filters (kernels). This is then followed by a non-linear activation function. The pooling layers reduce dimensionality while preserving key information. This ensures the model remains both complex yet efficient. Lastly, the output feature maps are flattened into one-dimensional arrays and passed through fully-connected layers, which combine this information to perform the final classification [31].

### 3.3.2 CNN-Attention

An attention-enhanced variation of the baseline CNN was also implemented. Attention introduces the ability to identify more complex patterns and improve feature weighting. Attention assigns higher weights to the most discriminative features [32], enabling the CNN to focus on more informative regions rather than treating all features equally. Both CNN variants support automatic feature extraction [31] and can generate complex feature maps, making them a positive option for an IDS designed for edge deployment.

# 4 Methodology

Building on current research into AI-based intrusion detection for IoT systems, this research follows the proposed pipeline in Figure 1 for training and deploying four AI models to detect malicious network traffic in IoT systems. This framework is designed to determine which AI architecture balances accuracy, efficiency, and edge deployability.

## 4.1 Data Preprocessing

Preprocessing of the RT-IoT2022 dataset comprises multiple, sequential steps: visualization, feature selection, encoding, and normalization. Histogram and heatmap visualizations are used to assess null values, feature distributions, and correlations. The target feature histogram shown in Figure 2 revealed the extreme imbalance of the dataset, highlighting several minority and ultra rare classes. These analyses help identify outliers, direct the feature selection process, and guide the following preprocessing steps.

The dataset is first split into an 80-20 train-test stratified split to prevent data leakage and ensure a proper test environment. Feature importance is then analyzed using mutual information (MI) scores and RF Classifier rankings to guide dimensionality reduction and improve model efficiency [33]. MI is a measure of the reduction in uncertainty of the target value given a feature $X$ [34]. Before calculating the MI scores, the discrete features were

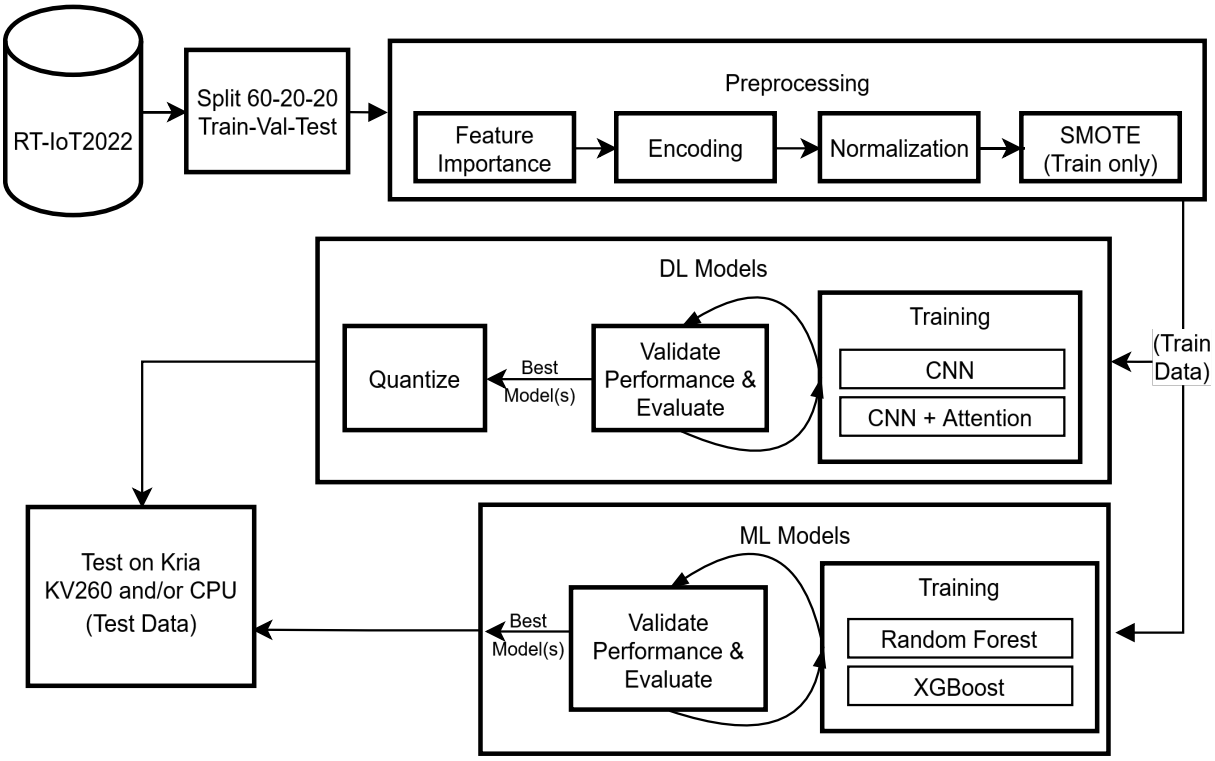


Figure 1: Process Pipeline Diagram

either withheld or marked as discrete in Scikit-learn's MI scoring function arguments. The port features, id.res_p and id.orig_p, were excluded from scoring solely to avoid estimator distortion due to their large domain range [1, 65535]. Their relevance is still captured through the RF importance analysis, which can account for high-cardinality discrete variables. Figure 3 shows the ten highest-scoring features from the MI score, ordered from most to least informative. This combination of MI and RF analysis reduces model complexity and supports efficient edge deployment without compromising predictive performance. Mutual information is chosen for its ability to detect nonlinear dependencies [34], while RF importance provides a complementary model-centric perspective and a secondary check of feature importance, including the port fields.

Next, categorical features, protocol and service, are encoded using one-hot encoding and label encoding, respectively. The protocol feature has three distinct categories, making one-hot encoding appropriate. In contrast, the service feature includes many unique values,

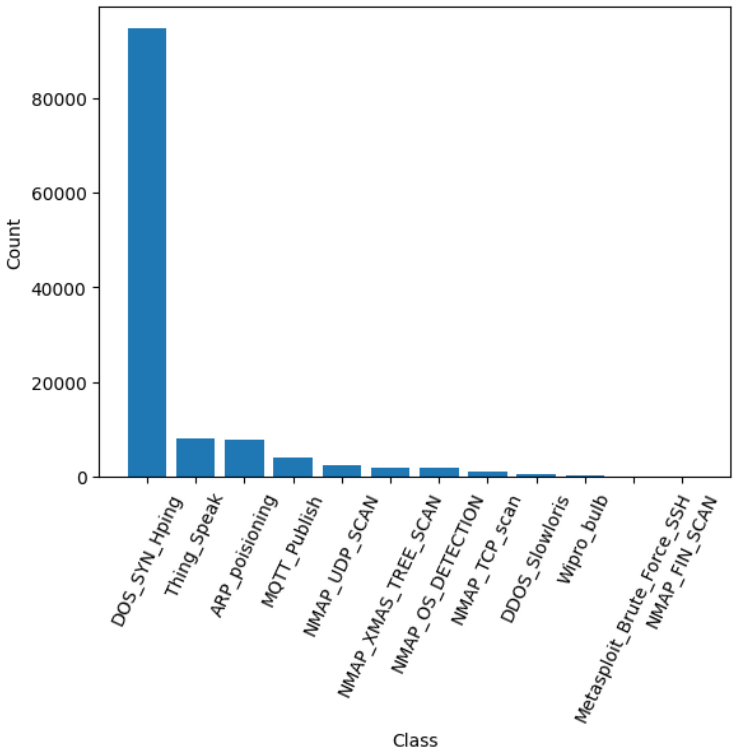


Figure 2: Target Class Distribution Histogram

meaning label encoding is applied to maintain a lower dimensionality while converting it into numerical form. This ensures not only all non-numerical variables are in the required format for model compatibility, but also facilitates subsequent data processing. Finally, the processed subset is normalized within the range [-1, 1] using a Min-Max scaling to maintain consistency across feature ranges and enhance training stability.

After data preparation, the dataset is split further into 80-20 train-validation sets for an overall data partition of 60-20-20, train-validation-test. This is to ensure proper training, hyperparameter tuning, and validation of the models. To mitigate the extreme dataset imbalance, the SMOTE algorithm [35] was applied to the multi-class training data for class balancing after partitioning out the test and validation sets. SMOTENC, a variant of SMOTE, accounts for categorical features, along with numerical features. For each new generated data point, the categorical features are sampled from the values of the k-nearest neighbors, thus preserving data realism. This technique increases the dataset size by synthetically generating

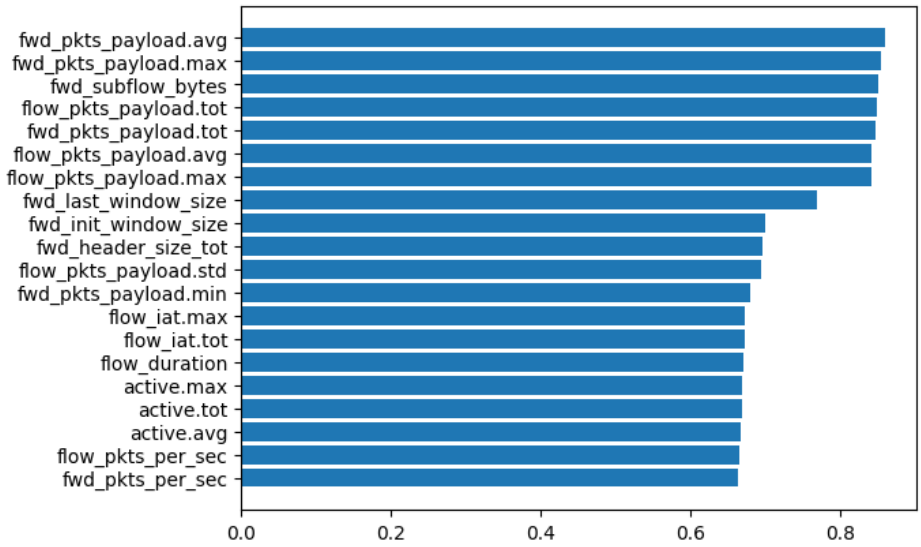


Figure 3: MI Scores: Top 10 Highest Scoring Features

new entries for minority classes according to the sampling strategy, thereby improving both accuracy and recall. Once preprocessed and oversampled, the data is prepped for training.

## 4.2 Models

For an in-depth comparison of different AI architectures and their impact on IDS design, the structure and rationale of the four selected models are outlined in this section. Two ensemble ML models and two DL CNN models are evaluated, with each trained separately for binary and multi-class classification objectives.

### 4.2.1 ML Models

Two ensemble ML models—RF and XGBoost—are selected for their strong baseline performance on tabular data and ability to recognize non-linear interactions. Additionally, their computational efficiency makes them suitable for comparison against the two DL models evaluated in this paper. A baseline version of each classifier was first created, with the objective function (binary or multi-class classification) set appropriately and class weighting applied to account for dataset imbalance. Model performance was then fine-tuned using a

randomized cross-validation (CV) grid search using 5-fold CV to set the remaining hyper-parameters. The highest scoring configurations were then used to train the final ML models for experimental evaluation.

### 4.2.2 DL Models

Two CNN configurations were evaluated: a baseline CNN and a CNN augmented with multi-headed self-attention. CNNs were chosen for their ability to find complex, hidden patterns within data, while attention mechanisms enable adaptive feature weighting and enhance minority-class recognition. The attention-supported variant was designed to improve the model's ability at identifying minority classes as CNNs can struggle to generalize to rare classes under severe class imbalance [36]. This challenge is particularly relevant for the RT-IoT2022 dataset, where the majority class, DOS_SYN_Hping, accounts for nearly 77% of all samples, and attack classes collectively constitute almost 90% of total entries as shown in Figure 2.

The model structures are outlined in Figure 4. Both networks comprise two two-dimensional convolutional layers followed by a ReLu activation function (1) and either an average-pooling or max-pooling layer to reduce spatial dimensionality. These pooling strategies were selected empirically based on validation performance.

$$f(x) = \max(0, x) \tag{1}$$

The output feature maps are then flattened and fed through a dense, fully-connected layer followed by another ReLu activation. A Dropout layer is included for regularization. The final fully-connected layer predicts the output, either two or twelve output classes, depending on the classification objective.

In this work, 2D-CNNs configured with a mock spatial height of one, are used to emulate 1D-CNN behavior by convolving across the feature dimension. This design enables

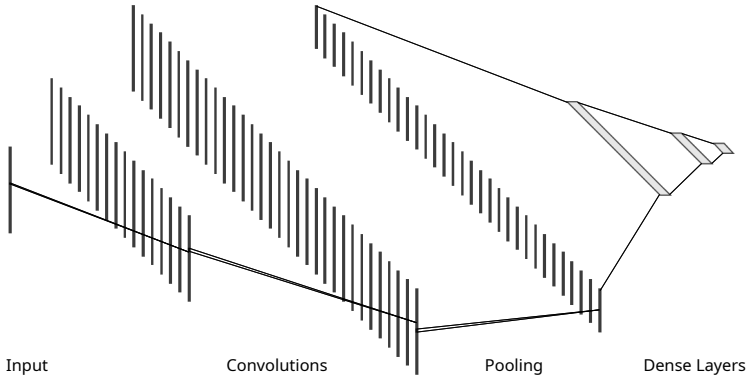


Figure 4: CNN Architecture

compatibility with the Kria KV260 hardware and Vitis-AI, which do not natively support 1D-CNN operators. These models are applied to tabular, flow-level features as a proof-of-concept (POC) for edge deployment. Prior studies have shown that 1D-CNNs can perform effectively for IoT intrusion detection [37–39]. The RT-IoT2022 dataset is non-sequential, contains no inherent feature ordering, and lacks temporal information. Consequently, temporal dependencies are not explicitly modeled, and the proposed CNN structure operates as a spatial feature extractor. While this approach may introduce artificial feature correlations, it is necessitated by the absence of temporal structure in the dataset. In real-world settings, however, network flow data inherently exhibits temporal correlations. Practical applications would therefore employ temporal sliding windows to enable convolution across time steps. Studies such as [38,39] highlight the benefits of incorporating temporal structure into 1D-CNN-based IDSs.

This work focuses on validating the proposed CNN structure in a non-temporal setting and its deployment viability onto an edge device, with temporal windowing reserved for future research. As a result, the structure of the input data is set to have a sequence length of one, with the convolutions operating across the feature dimension. This design allows the evaluation of feasibility on resource-constrained hardware.

For the CNN-Attention model, a multi-headed self-attention block is incorporated after the pooling layer and before the dense network. To do so, the pooled feature maps are reshaped into a sequence of tokens, enabling the attention mechanism to learn contextual relationships before classification.

## 4.3 Performance Analysis

Each model undergoes an individual quantitative analysis using key performance metrics: accuracy, precision, recall, F1-Score, and Matthew's correlation coefficient (MCC), as well as a confusion matrix. These metrics collectively assess how well the model distinguishes classes, with particular attention at minimizing false positives (FP) and false negatives (FN), as both pose significant detriment to an IDS. If even a single attack is missed, the entire system may be compromised. To provide a consistent basis for comparison, each metric is formally defined below.

Accuracy is a measure of the model's overall correctness and is defined by Formula (2). True positives (TP) denote the number of correctly identified positive samples. Similarly, true negatives (TN) represent correctly identified negative samples. FP denotes samples identified as positive, but the ground truth is negative. Likewise, FN represents incorrectly identified samples as negative when the ground truth is positive. In a multi-class setting, TP corresponds to the number of correctly classified instances belonging to class $k$, and TN represents the total number of correctly classified samples belonging to all other classes. Comparably, FP corresponds to the number of instances incorrectly assigned to class $k$, and FN is the total number of samples that truly belong to class $k$ but are incorrectly classified otherwise.

$$\text{Accuracy} = \frac{TP + TN}{\text{Total Samples}} \tag{2}$$

Precision measures the reliability of the positively identified classes, that is, among the positive predictions, how many of them were actually positive. This is defined in Formula

(3).

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$

Recall is especially important in measuring the performance of a network IDS. It measures how effectively the model identified instances of the minority, or negative, classes and is defined in Formula (4).

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4}$$

The F1-Score combines these metrics into one balanced measure that computes the harmonic mean between precision and recall, as shown in Formula (5).

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{5}$$

The confusion matrix is a $k$-dimensional matrix that records predicted versus true class distributions, with diagonal entries representing correctly classified samples. The confusion matrix is used to visualize the per-class classification ability of the model and helps identify the model's weak points.

MCC applies to both binary and multi-class classification tasks [40]. It is especially useful for imbalanced datasets, because it incorporates all entries of a confusion matrix rather than relying on per-class averages that are prone to majority-class dominance. MCC calculates a correlation value in the range [-1, 1], where 1 indicates perfect prediction and class separation of samples, -1 indicates consistent incorrect predictions but with correct separation, and 0 indicates no meaningful predictive performance. Furthermore, in binary classification settings, it is known that a high MCC score corresponds to a high AUC-ROC score [41]. Formula (6) shows this mathematically.

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{6}$$

For binary classification, these metrics were evaluated per class. For multi-class classifica-

tion, both the macro and weighted averages were computed. Macro averaging assigns equal weight to each class regardless of support values; therefore, misclassifications in minority classes heavily impact the score. Conversely, weighted averages scale each class according to its support, providing a more representative measure of overall performance. Both measures offer useful insights: the weighted average reflects general model effectiveness, whereas the macro average highlights when the model misses samples from the minority classes. This is an essential consideration in IDSs, where rare attacks can be just as harmful as common ones.

Per-sample latency is also measured to evaluate detection speeds in a deployment scenario. This is calculated according to Formula (7), where $T_i$ is the inference time of a single sample chosen from the dataset. This is repeated for 5000 iterations to obtain $\bar{T}$, an average per-sample latency time.

$$\bar{T}_{\text{sec}} = \frac{1}{5000} \sum_{i=1}^{5000} T_i \tag{7}$$

Following individual model evaluation, a comparative analysis aims to discover which among the four models best balances computational efficiency and classification performance for potential edge deployment. Overall hardware compatibility is compared alongside the previously obtained metrics to investigate this. The model that best balances across all metrics is selected for further development.

## 4.4 Edge Deployment

The chosen models are optimized and subsequently deployed for real-world testing and inference on an edge device, the Xilinx Kria KV260 Vision AI Starter Kit, to demonstrate real-world viability under resource constraints. This board features a Deep Processing Unit (DPU) implemented on a field-programmable gate array (FPGA), enabling customizable hardware acceleration for AI models [42]. While hardware acceleration is commonly performed using GPUs, acceleration on an FPGA device presents a unique research area for

comparison, as custom hardware acceleration can be optimized specifically for AI models and frequently process them even faster. The most compatible model from the DL models is selected for acceleration. In this study, the ML models are not quantized due to their incompatibility with the on-board DPU and designing a custom FPGA overlay is outside the scope of this research.

The best-balanced CNN model is quantized and compiled using the Vitis AI pipeline. Quantization transforms the floating-point representation of the trained model into fixed-point arithmetic. Although this generally results in a slight loss to precision, the fixed nature of the model considerably speeds up performance. After quantization, it is compiled and deployed on the DPU for accelerated inference using PYNQ software. The results from this process demonstrate the practical feasibility of running an AI-based IDS on an edge device, like the Kria KV260.

# 5 Experiment

To evaluate the effectiveness of four AI models for IoT-edge intrusion detection, a series of controlled experiments was conducted using both binary and multi-class classification objectives.

## 5.1 Experimental Setup

The experiments were performed on a preprocessed RT-IoT2022 dataset, which contains both benign and malicious traffic further categorized into types, such as NMAP scans and DDoS attacks. Preprocessing, described in Section 4.1, included visualization, stratified splitting, feature importance ranking, encoding, normalization, and SMOTE class balancing with consideration for categorical features. This step ensures uniformity in training the IDS algorithms.

Although the benign-malicious distribution of data is imbalanced at around a 90-10 split,

both the ML and DL models performed well without the use of SMOTE. Therefore, class balancing was not applied for binary classification experiments outside of setting weights in the hyperparameters.

The multiclass setting, however, exhibited far more severe imbalance. The dataset contains multiple minority classes with varying degrees of rarity, with the smallest class, NMAP_FIN_SCAN, having merely 28 samples (less than 0.023% of the dataset). To address this, a per-class SMOTENC sampling strategy was employed. The majority class, DOS_SYN_Hping, received no additional samples, while the remaining classes were oversampled based on different target levels rather than a single global ratio. A tiered sampling scheme was chosen because applying the same oversampling ratio to all classes would excessively inflate the smallest classes and introduce synthetic noise. By adjusting oversampling intensity based on each class's imbalance severity, the method increased minority-class representation while maintaining the relative structure of class distributions.

## 5.2 Training Procedure

After SMOTE resampling, the four models were each trained separately for both objectives. The models were divided into traditional ML and DL categories. RF and XGBoost were implemented as the traditional ML models and optimized using hyperparameter grid search. The resulting optimal parameters, summarized in Tables 1 and 2, were used for final model training.

The remaining two model architectures were DL-based—a baseline CNN and a CNN augmented with self-attention. All models operate on an input of (1x1×83), where the sequence length and height dimensions are one, leaving the model to convolve across the features.

The baseline CNN consists of two convolution layers (16 and 32 filters, kernel size (1,3)) each followed by ReLu activations. Binary models use max pooling before the dense layers; multi-class models use average pooling. The resulting feature maps are flattened and passed

Table 1: RF Model Hyperparameters

| Parameter | Binary | Multi |
|---|---|---|
| n_estimators | 200 | 100 |
| class_weight | balanced | balanced |
| min_samples_leaf | 1 | 2 |

Table 2: XGBoost Model Hyperparameters

| Parameter | Binary | Multi |
|---|---|---|
| n_estimators | 200 | 100 |
| objective | binary:logistic | multi:softprob |
| scale_pos_weight | $\frac{N_{negative}}{N_{positive}}$ | None |
| colsample_bytree | 1.0 | 0.8 |
| learning_rate | 0.3 | 0.1 |

through fully connected layers with dropout for regularization. The output is sized either 2 units or 12, depending on the task. The attention-augmented model incorporates a multi-headed self-attention block with 8 heads after the convolutional and pooling layers. The remaining dense network is the same as the baseline.

All DL model architectures were trained with the Adam optimizer, cross-entropy loss function, and a learning rate of $1 \times 10^{-4}$ on batches of 64 for up to 50 epochs. After validating on the validation set, the models were then ready for testing.

## 5.3 Evaluation Metrics

Model performance was tested using key performance metrics explained in Section 4.3, including accuracy, precision, recall, F1-score and MCC. For the multi-class experiments, both macro and weighted averages were reported to assess the model's performance on minority classes. Confusion matrices were generated to display per-class behavior, particularly for the CNN-Attention models as it was expected to better generalize to the minority classes. Beyond predictive performance, per-sample latency was calculated on the edge and the local workstation for inference timing.

### 5.4 Edge Deployment

To asses edge feasibility, the baseline DL models, chosen based on DPU compatibility and performance, were deployed to the Xilinx Kria KV260 following the standard Vitis AI workflow for quantization and compilation. The compiled models were executed on the on-board DPU, where an inference script measured per-sample latency and overall performance metrics. For reference, the same CNN architecture was also timed on the workstation's i7 CPU. As designing a custom FPGA overlay is outside the scope of this work, the best ML models were executed only on the CPU for measurement of the same metrics.

### 5.5 Experimental Conditions

All training and workstation testing was performed on a local workstation with an Intel Core i7 processor and 32 GB of RAM. Edge testing was conducted using a Xilinx Kria KV260 running Ubuntu 22.04 and PYNQ for Kria. These models were implemented using Python 3.12 and libraries Scikit-learn and XGBoost for ML and PyTorch for DL.

## 6 Results

In this section, the results of the RF, XGBoost, CNN, and CNN-Attention models are presented and discussed. The feasibility of edge deployment is also demonstrated in this section.

### 6.1 Binary Classification Results

For binary classification, Table 3 details the performance metrics of each classifier, where precision, recall, and F1 correspond to the positive (malicious) class values. Among the ML models, RF performed the best with an F1-Score of 99.91% and an MCC value of 99.13%, indicating high class separability. XGBoost, the other ML model tested, demonstrated comparable performance matching the F1 with only marginal reductions across the remaining

metrics. These results confirm the strong suitability of tree-based models as an IDS for binary tasks on the evaluated dataset.

Table 3: Binary Classification

| Metric | RF | XGBoost | CNN | CNN + Attn |
|---|---|---|---|---|
| Accuracy | 0.9984 | 0.9983 | 0.9913 | 0.9900 |
| MCC | 0.9913 | 0.9909 | 0.9522 | 0.9443 |
| Precision | 0.9988 | 0.9994 | 0.9940 | 0.9911 |
| Recall | 0.9994 | 0.9987 | 0.9964 | 0.9978 |
| F1 | 0.9991 | 0.9991 | 0.9952 | 0.9945 |

The DL models also performed well. Although the unordered nature of the features limits the theoretical suitability of convolutional operators, the CNN models still achieved competitive empirical performance. The baseline CNN achieved an F1 of 97.61%, while the attention variant reached 99.45%, just short of the ML classifiers' performance. This suggests that, in practice, the learned feature interactions remain discriminative for the task, even without explicit spatial or temporal structure. Importantly, unlike the tree-based models, the baseline CNN can be fully quantized and deployed on the target DPU, enabling real-time inference at the edge. This highlights the practical trade-off between peak classification performance and deployability under strict hardware constraints.

## 6.2 Multi-Class Classification Results

For the multi-class experiments, the quantitative results for all models are reported in Table 4. The ML classifiers again achieved the strongest overall performance. XGBoost obtained the highest macro F1-Score, 97.92% and an MCC of 99.51%, with RF performing at a similar level, only slightly lower. These results further demonstrate the effectiveness and stability of tree-based ensemble methods for multi-class intrusion detection on this dataset.

The DL models, while not surpassing the ML classifiers, still demonstrated competitive results. As in the binary case, the lack of inherent feature ordering limits the theoretical alignment between convolutional operations and underlying data geometry, potentially caus-

Table 4: Multi-Class Classification

| Metric | RF | XGBoost | CNN | CNN + Attn |
|---|---|---|---|---|
| Accuracy | 0.9980 | 0.9981 | 0.9493 | 0.9905 |
| MCC | 0.9949 | 0.9951 | 0.8753 | 0.9762 |
| *Weighted Average* | | | | |
| Precision | 0.9980 | 0.9981 | 0.9391 | 0.9924 |
| Recall | 0.9980 | 0.9981 | 0.9493 | 0.9905 |
| F1 | 0.9980 | 0.9980 | 0.9406 | 0.9912 |
| *Macro Average* | | | | |
| Precision | 0.9469 | 0.9962 | 0.5073 | 0.8536 |
| Recall | 0.9682 | 0.9645 | 0.6240 | 0.9351 |
| F1 | 0.9569 | 0.9792 | 0.5324 | 0.8686 |

ing the model to learn artificial local correlations. Nevertheless, the baseline CNN achieved strong weighted performance, with a weighted F1 of 94.06%. The CNN-Attention model performed much better at identifying minority classes than the baseline, as expected. The macro F1 had a value of 86.86%, more than 30 percentage points better than the baseline. While the macro F1 reflects sensitivity to minority class errors, the weighted F1 of 99.12% indicates high overall classification performance when accounting for class imbalance.

A deeper analysis into the misclassified samples shows that the CNN models struggle with discriminating the NMAP scan subclasses. The dataset contains five distinct NMAP varieties. As shown in the baseline CNN confusion matrix in Figure 5, four of these subclasses could not be found at all, rather nearly all mapped to the NMAP_UDP_SCAN class. This indicates the model is able to reliably identify NMAP scans in general, but it fails to distinguish the scan type. The CNN-Attention variant substantially improved subclass discrimination as shown in Figure 6, successfully identifying all five NMAP subclasses. This suggests that by adding attention, the network is able to apply more weight to the subset of features that distinguish NMAP scans and properly classify the types.

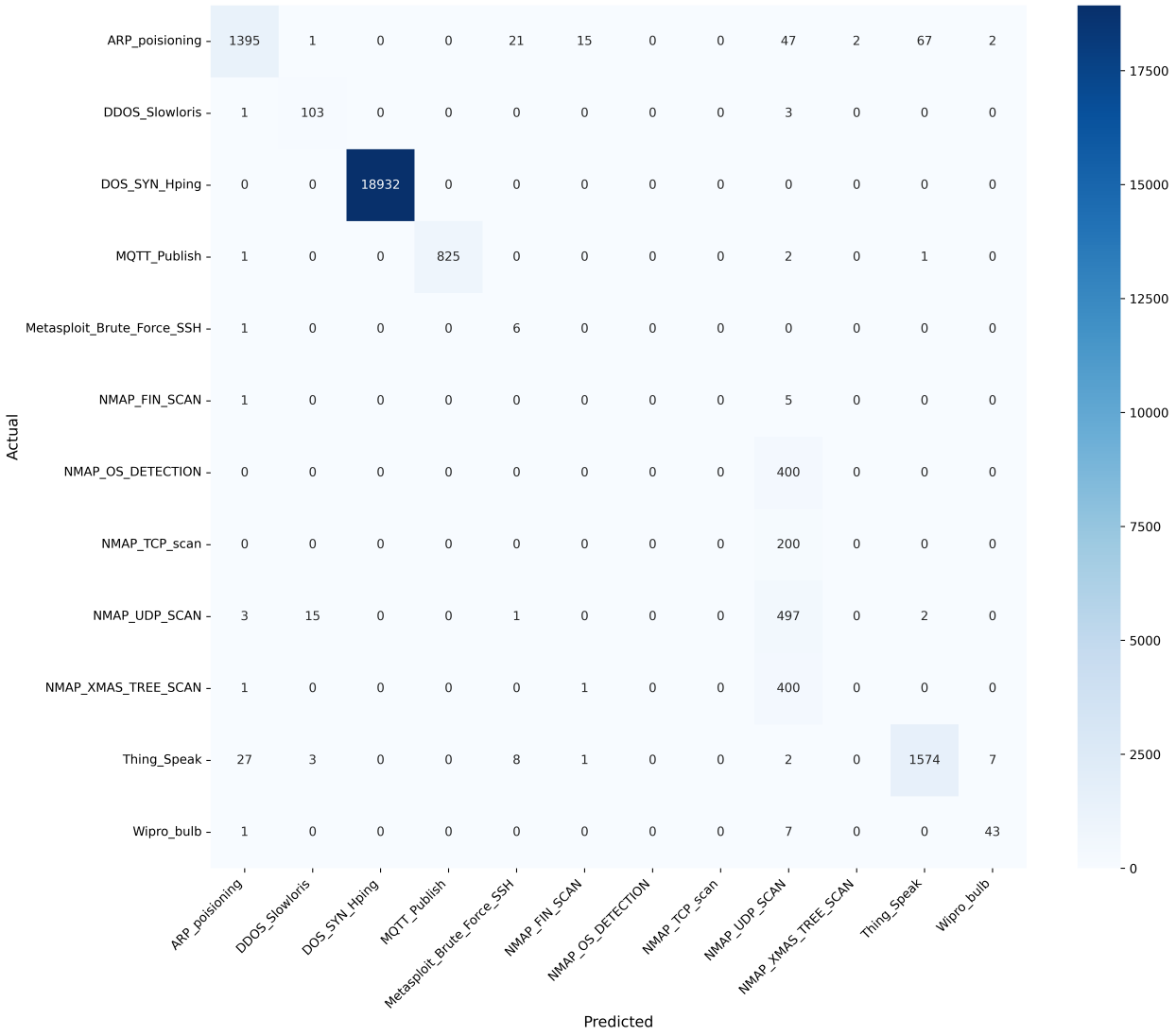


Figure 5: Multi-Class Baseline CNN Confusion Matrix

## 6.3 Edge Deployment Results

For determining edge performance, the per-sample latency of the accelerated models was measured. Figure 7 reports the per-sample latency of three binary classifiers: one DL model and both ML models. The baseline CNN was selected for quantization and deployment, due to its compatibility with Vitis-AI and the Kria DPU. It was benchmarked on both the Kria and the Intel i7 CPU. Both ML models were chosen for reference testing on the i7 CPU.

As expected, the CNN running on the DPU was significantly faster than all other configurations, with the lowest latency at 0.18 milliseconds (ms) per sample, representing a more

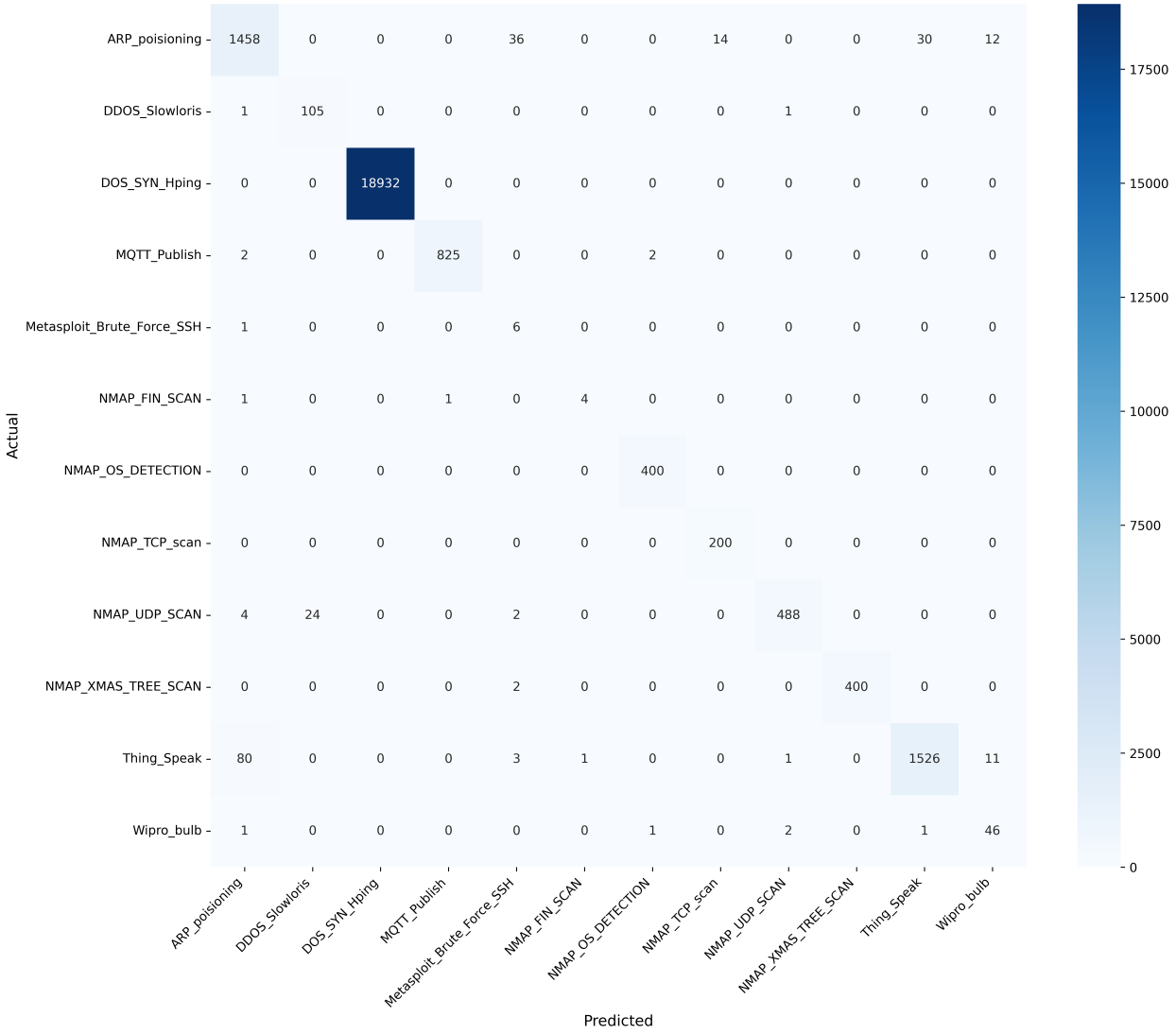


Figure 6: Multi-Class CNN-Attention Confusion Matrix

than 2.5× speedup over its CPU counterpart. Both ML models had considerably higher latency, in particular, RF had a time of 10.73 ms, over 59× slower than the accelerated DL model.

The results follow a similar trend for multi-class classification in Figure 8 when comparing the baseline CNN to both ML models. The baseline CNN running on the DPU achieved a latency of 0.21 ms per sample, compared to 0.71 ms on the Intel i7 CPU, a more than 3.25× slowdown. Both ML models, again, performed much slower. XGBoost was the slowest, taking over 35× longer per sample than the accelerated CNN. In the ML models,

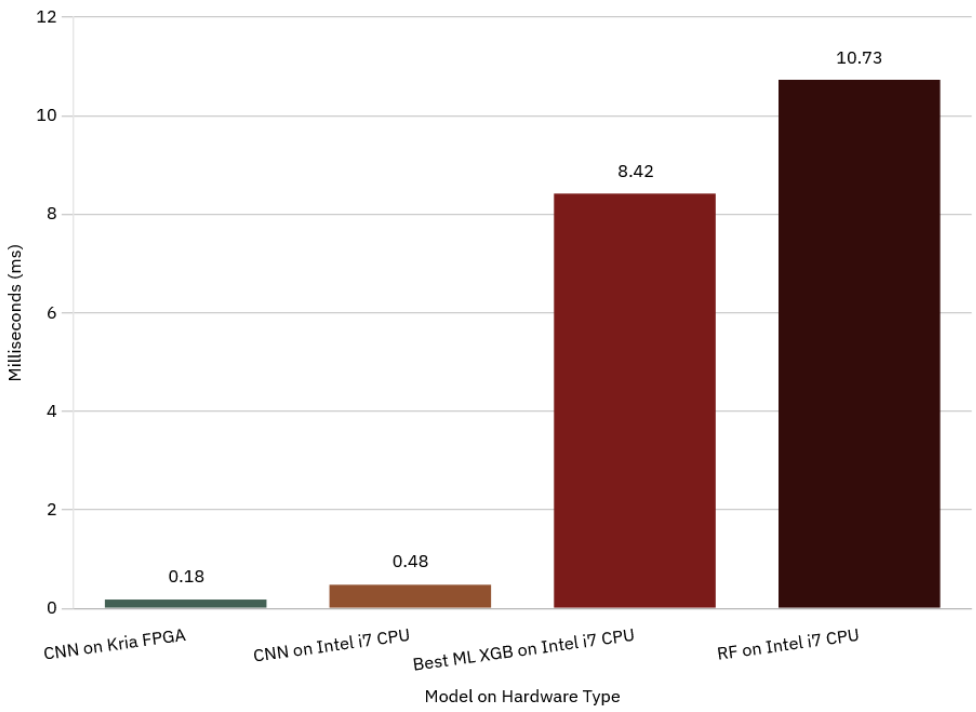


Figure 7: Binary Models' Per Sample Latency Across Different Hardware

the reduced number of estimators in the multi-class configuration led to noticeably faster inference compared to their binary counterparts.

While the accelerated CNNs show significant reductions in inference time, it is necessary to analyze the predicted results to determine whether the lower latency justifies any performance trade-offs. For the binary CNN on the FPGA, the precision and recall for the positive (malicious) class increased by 0.18 and 0.07 percentage points, respectively. However, the F1-Score decreased by 0.38 percentage points, indicating a slight overall performance drop. A closer examination reveals that the model performed notably worse on the benign class, but due to the strong class imbalance, the decline was not strongly observed in its F1-Score. The MCC score, more reflective of all confusion matrix entries, was 91.95%, a 3.27 percentage point reduction, indicating reduced class separability. The minimal reduction in performance indicates the model's suitability for inference on an FPGA.

For the multi-class accelerated CNN, performance increased substantially. The macro F1 rose from 53.24% (CPU baseline) to 83.23% after quantization, a nearly 30-point im-

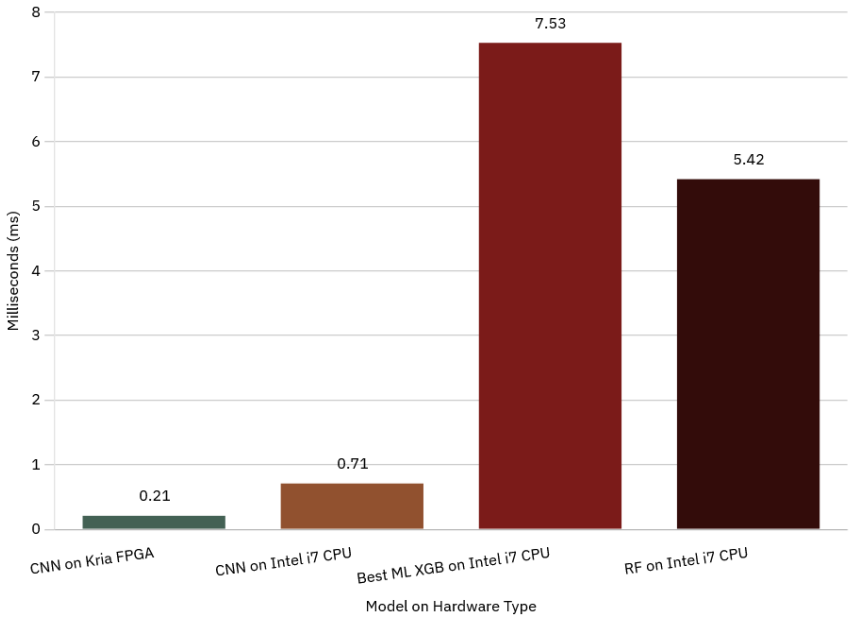


Figure 8: Multi-Class Models' Per Sample Latency Across Different Hardware

provement. This suggests that quantization effectively simplified the model and reduced overfitting, indicating that the original model was too complex for this particular dataset. These findings highlight the need for further study on how quantization affects multi-class behavior in this domain, particularly to assess whether the resulting trade-offs are acceptable.

Overall, these results demonstrate the practicality and advantages of deploying accelerated DL models on edge hardware for intrusion detection. The significant reduction in latency enables near real-time threat detection on resource-constrained IoT devices, while the limited performance degradation, if any, suggests that the trade-offs introduced by quantization are minimal. Further research in this area could investigate the use of more realistic datasets or quantization-aware training to minimize the adverse effects. The successful proof of concept further demonstrates the feasibility of FPGA-based IDS deployment and supports continued exploration into optimized edge-focused DL approaches.

# 7 Limitations and Future Directions

This work also acknowledges several limitations. For both binary and multi-class classification, more robust datasets that better represent real-world IoT network traffic are needed to further validate results. Additionally, the dataset's lack of temporal information limited the ability to leverage the strengths of the CNN architecture, which typically benefit from sequential information inherent to network flows. Incorporating temporal features may enable the model to better learn traffic patterns and improve performance on rare classes. Evaluating alternative architectures with stronger quantization compatibility, along with comparing different edge hardware platforms, would provide additional insight into practical deployment choices. Finally, due to time constraints, only the baseline CNN models were tested on the DPU. Evaluating alternative architectures with stronger quantization compatibility, along with comparing different edge hardware platforms, would provide additional insight into practical deployment choices.

Future researchers could build on this study by applying the same evaluation framework to more complex, realistic datasets or by designing a lightweight data ingestion and transformation pipeline suitable for edge environments. Exploring hybrid DL architectures, which have already shown promising performance in this domain [22–25], also represents a valuable direction to explore. Altogether, these avenues highlight how continued refinement of both data and model design can further strengthen the effectiveness of edge-based IDS systems.

# 8 Conclusion and Discussion

Securing resource-constrained IoT networks remains challenging due to evolving cyber threats and IoT device diversity and ubiquity. This research investigated AI-powered intrusion detection systems for IoT networks, spanning both ML and DL methodologies. The models used in this study were RF, XGBoost, CNN, and CNN with self-attention. Each model was benchmarked against the RT-IoT2022 dataset for binary and multi-class objectives, mea-

suring performance using metrics such as F1-Score, MCC, and latency. The best-balanced model was deployed and accelerated to act as a guardian on an edge device for real-time threat detection, demonstrating the feasibility of an IDS implemented at the edge.

For binary classification, the RF ensemble model achieved the best performance with an F1-Score of 99.91%, showing its strong capability in classifying IoT network traffic. The best binary DL model, the baseline CNN, achieved a comparable F1 of 99.52%. When accelerated and deployed to the edge on a Kria KV260, the inference speed of the baseline CNN increased by over 2.5 times from CPU performance, while the drop in precision was marginal, with an F1 of 99.14%. The successful deployment on an FPGA demonstrated the benefits edge computing and model acceleration bring to IDS design.

In a multi-class setting, the ML models outperformed all other models with XGBoost having a macro F1 of 97.92%, highlighting its suitability for applications in intrusion detection systems. Due to the heavily imbalanced dataset, the DL models had difficulty generalizing and separating less distinct classes despite the use of SMOTE class balancing during training. Among the DL configurations tested, the CNN-Attention variant performed best with a macro F1 of 86.86%. Despite the lower score, the CNN-Attention model still performed well overall, with a weighted F1-Score of 99.12% and MCC of 97.62%, signifying its strong classification separability. The lower macro F1 score was primarily due to missing a few instances of the smaller classes in the dataset. Self-attention demonstrated clear benefits by correctly discriminating between most of the minority classes (NMAP scan subtypes), unlike its baseline counterpart.

The feasibility of real-world edge deployment for multi classification was also demonstrated through a working proof of concept of the baseline CNN. The speedup offered by the accelerated model was over 3.25x faster than execution on a CPU. The performance of the accelerated model was higher by a significant margin, suggesting that the baseline CNN model was too complex and overfit on this particular dataset. Therefore further research would involve examining the affects of acceleration on other datasets. The accelerated models

illustrated the benefits FPGAs offer in the IoT cybersecurity domain. These improvements show that edge acceleration can meaningfully reduce inference latency without sacrificing much model performance.

The successful application of an AI-based IDS at the edge serves as a reference for future IoT IDS edge implementations. This research further contributes insights to IDS model comparative studies by systematically evaluating both ML and DL approaches across binary and multi-class objectives. It also provides one of the few explorations into deploying an AI-based IDS on an FPGA-equipped device, helping bridge the gap between simulation-based research and real-world deployment. Overall, this is a very strong study for application for edge devices in securing IoT systems.

# Biography

Willow Connelly is a senior undergraduate student at Oakland University pursuing a Bachelor's degree in Computer Science. Because of coursework in these fields, she has developed a strong interest in data science, AI technologies, and quantum computing. This academic background has inspired her to expand her educational goals, with plans to ultimately pursue a graduate degree that encompasses these disciplines.

Connelly has completed two internships with General Motors in different areas, one of which focused specifically on data mining, modeling, and visualization. Following that experience, her interest in that sector grew and she has set her sights on seeking a career in researching AI-driven solutions. This thesis project, which investigates the efficacy of various AI models as a cybersecurity solution in a complex, interconnected IoT environment, aligns with her goal of exploring AI in different contexts. Through this work, she has demonstrated her research abilities, data analysis skills, and programming expertise. Additionally, it has supported her future plans by providing hands-on understanding in designing AI systems for unique circumstances and giving experience in writing a formal thesis.

## Acknowledgments

This project did not involve human subject research and did not require IRB approval.

The author would like to thank Dr. Kate Bowers for their guidance and support throughout the duration of this research. Her introduction to Edge AI through her class inspired this project, and her support with providing the Kria platform was invaluable during the course of the project.

## References

[1] S. Greengard, *The Internet of Things, Revised and Updated Edition.* Cambridge, MA, USA: The MIT Press, 2021.

[2] B. Ji, X. Zhang, S. Mumtaz, C. Han, C. Li, H. Wen, and D. Wang, "Survey on the internet of vehicles: Network architectures and applications," *IEEE Communications Standards Magazine*, vol. 4, no. 1, pp. 34–41, 2020.

[3] Transforma Insights' TAM Forecasts, "IoT connections forecast 2023-2034," 2025, Last Accessed: 2025-11-15. [Online]. Available: https://transformainsights.com/research/forecast/highlights

[4] R. Vallabhaneni, "Effects of data breaches on internet of things (IoT) devices within the proliferation of daily-life integrated devices," *Engineering And Technology Journal*, vol. 9, no. 7, pp. 4439–4442, 2024.

[5] N. S. Abouzakhar, A. Jones, and O. Angelopoulou, "Internet of things security: A review of risks and threats to healthcare sector," in *2017 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2017, pp. 373–378.

[6] V. Hassija, V. Chamola, V. Saxena, D. Jain, P. Goyal, and B. Sikdar, "A survey on IoT security: Application areas, security threats, and solution architectures," *IEEE Access*, vol. 7, pp. 82 721–82 743, 2019.

[7] M. M. Rahman, S. A. Shakil, and M. R. Mustakim, "A survey on intrusion detection system in IoT networks," *Cyber Security and Applications*, vol. 3, p. 100082, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772918424000481

[8] ——, "A survey on intrusion detection system in iot networks," *Cyber Security and Applications*, vol. 3, p. 100082, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772918424000481

[9] A. Hozouri, A. Mirzaei, and M. Effatparvar, "A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges," *Discover Artificial Intelligence*, vol. 5, no. 314, 2025. [Online]. Available: https://doi.org/10.1007/s44163-025-00578-1

[10] B. Sharmila and R. Nagapadma, "RT-IoT2022," UCI Machine Learning Repository, 2023, DOI: https://doi.org/10.24432/C5P338.

[11] M. A. O. Ahmed, Y. AbdelSatar, R. Alotaibi, and O. Reyad, "Enhancing internet of things security using performance gradient boosting for network intrusion detection systems," *Alexandria Engineering Journal*, vol. 116, pp. 472–482, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1110016824017046

[12] P. K. Danso, E. C. P. Neto, S. Dadkhah, A. Zohourian, H. Molyneaux, and A. A. Ghorbani, "Ensemble-based intrusion detection for internet of things devices," in *2022 IEEE 19th International Conference on Smart Communities: Improving Quality of Life Using ICT, IoT and AI (HONET)*, 2022, pp. 034–039.

[13] N. Jeffrey, Q. Tan, and J. R. Villar, "Using ensemble learning for anomaly detection in cyber–physical systems," *Electronics*, vol. 13, no. 7, 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/7/1391

[14] G. De Carvalho Bertoli, L. A. Pereira Júnior, O. Saotome, A. L. Dos Santos, F. A. N. Verri, C. A. C. Marcondes, S. Barbieri, M. S. Rodrigues, and J. M. Parente De Oliveira, "An end-to-end framework for machine learning-based network intrusion detection system," *IEEE Access*, vol. 9, pp. 106 790–106 805, 2021.

[15] D. C. Attota, V. Mothukuri, R. M. Parizi, and S. Pouriyeh, "An ensemble multi-view federated learning intrusion detection for IoT," *IEEE Access*, vol. 9, pp. 117 734–117 745, 2021.

[16] E. Hallaji, R. Razavi-Far, and M. Saif, "Expanding analytical capabilities in intrusion detection through ensemble-based multi-label classification," *Computers & Security*, vol. 139, p. 103730, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404824000312

[17] I. A. Fares, M. Abd Elaziz, A. O. Aseeri, H. S. Zied, and A. G. Abdellatif, "TFKAN: transformer based on Kolmogorov–Arnold networks for intrusion detection in IoT environment," *Egyptian Informatics Journal*, vol. 30, p. 100666, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1110866525000593

[18] X. Yu, Y. Huang, Y. Zhang, M. Song, and Z. Jia, "Network intrusion traffic detection based on feature extraction," *Computers, Materials &Continua*, vol. 78, no. 1, pp. 473–492, 2024. [Online]. Available: http://www.techscience.com/cmc/v78n1/55387

[19] M. Tayebi and S. El Kafhali, "Performance analysis of recurrent neural networks for intrusion detection systems in industrial-internet of things," *Franklin Open*, vol. 12, p. 100310, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2773186325000982

[20] B. Roy and H. Cheung, "A deep learning approach for intrusion detection in internet of things using bi-directional long short-term memory recurrent neural network," in *2018 28th International Telecommunication Networks and Applications Conference (ITNAC)*, 2018, pp. 1–6.

[21] A. Waqas, S. D. Khan, Z. Ullah, M. Ullah, and H. Ullah, "Comparative analysis of deep learning models for intrusion detection in iot networks," *Computers*, vol. 14, no. 7, 2025. [Online]. Available: https://www.mdpi.com/2073-431X/14/7/283

[22] J. Du, K. Yang, Y. Hu, and L. Jiang, "NIDS-CNNLSTM: Network intrusion detection classification model based on deep learning," *IEEE Access*, vol. 11, pp. 24 808–24 821, 2023.

[23] M. A. Wahed, "Real-time intrusion detection and traffic analysis using AI techniques in IoT infrastructure," in *2024 1st International Conference on Emerging Technologies for Dependable Internet of Things (ICETI)*, 2024, pp. 1–6.

[24] A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "CNN-LSTM: hybrid deep neural network for network intrusion detection system," *IEEE Access*, vol. 10, pp. 99 837–99 849, 2022.

[25] B. M. Elzaghmouri, Y. H. Fayez Jbara, S. Elaiwat, N. Innab, A. A. F. Osman, M. A. M. Ataelfadiel, F. H. Zawaideh, M. F. Alawneh, A. Al-Khateeb, and M. Abu-Zanona, "A novel hybrid architecture for superior IoT threat detection through real IoT environments," *Computers, Materials and Continua*, vol. 81, no. 2, pp. 2299–2316, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1546221824007963

[26] A. Singh, "Real time intrusion detection in edge computing using machine learning techniques," *Turkish Journal of Engineering*, vol. 9, no. 2, p. 385–393, 2025.

[27] T. Alladi, V. Kohli, V. Chamola, F. R. Yu, and M. Guizani, "Artificial intelligence (AI)-empowered intrusion detection architecture for the internet of vehicles," *IEEE Wireless Communications*, vol. 28, no. 3, pp. 144–149, 2021.

[28] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[29] S. Uddin and H. Lu, "Confirming the statistically significant superiority of tree-based machine learning algorithms over their counterparts for tabular data," *PLoS One*, 2024.

[30] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[31] L. Alzubaidi, J. Zhang, A. J. Humaidi, and et al., "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," *J Big Data*, vol. 8, no. 53, 2021.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[33] R. Battiti, "Using mutual information for selecting features in supervised neural net learning," *IEEE Transactions on Neural Networks*, vol. 5, no. 4, pp. 537–550, 1994.

[34] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ, USA: Wiley-Interscience, 2006.

[35] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, p. 321–357, 2002.

[36] D. Dablain, C. B. Kristen N. Jacobson, M. Roberts, and N. V. Chawla, "Understanding CNN fragility when learning with imbalanced data," *Machine Learning*, vol. 113, pp. 4785–4810, 2024.

[37] M. Azizjon, A. Jumabek, and W. Kim, "1D CNN based network intrusion detection with normalization on imbalanced data," in *2020 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*, 2020, pp. 218–224.

[38] E. U. H. Qazi, A. Almorjan, and T. Zia, "A one-dimensional convolutional neural network (1d-cnn) based deep learning system for network intrusion detection," *Applied Sciences*, vol. 12, no. 16, 2022. [Online]. Available: https://www.mdpi.com/2076-3417/12/16/7986

[39] M. Arslan, M. Mubeen, M. Bilal, and S. F. Abbasi, "1D-CNN-IDS: 1D CNN-based intrusion detection system for IIoT," 2024. [Online]. Available: https://arxiv.org/abs/2409.08529

[40] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classification: an overview," 2020. [Online]. Available: https://arxiv.org/abs/2008.05756

[41] D. Chicco and G. Jurman, "The matthews correlation coefficient (MCC) should replace the ROC AUC as the standard metric for assessing binary classification." *BioData Mining*, vol. 16, no. 4, 2023. [Online]. Available: https://doi.org/10.1186/s13040-023-00322-4

[42] Advanced Micro Devices, Inc., "Kria KV260 Vision AI starter Kit User Guide," https://docs.amd.com/r/en-US/ug1089-kv260-starter-kit/Summary, Advanced Micro Devices, Inc., 2021, accessed: 2025-12-02.